Neurocomputing 666 (2026) 132226

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Lightweight AI-driven traffic forecasting and shaping for 6G LEO satellite networks

Yu Liang [a,b,c], Mingji Dong [a,b,c], Xiaofan Xu [a,b,c,*], Ping Du [a,b,c,*], Maozhen Li [d], Zuqing Zhu [e]

[a] Shanghai Satellite Network Research Institute Co. Ltd., Shanghai 200120, China
[b] State Key Laboratory of Satellite Network, Shanghai 200120, China
[c] Key Laboratory of Satellite Network, Shanghai 200120, China
[d] Department of Electronic and Electrical Engineering, Brunel University London, Uxbridge, UB8 3PH, UK
[e] School of Information Science and Technology, University of Science and Technology of China, Hefei, Anhui 230027, China

ARTICLE INFO

Communicated by N. Zeng

Keywords:
Low earth orbit satellite networks
iTransformer
Credit-based shaping
Time series prediction
Resource allocation

ABSTRACT

Low Earth Orbit (LEO) satellite networks are expected to be a key enabler of 6G communications, providing global coverage and low-latency services for remote and underserved regions. However, their dynamic topologies, large-scale deployments, and limited onboard resources pose significant challenges to reliable service delivery. This paper presents a lightweight, AI-driven framework for service flow forecasting in LEO networks to minimize latency and ensure compliance with service level agreements (SLAs). Our main contributions are: (1) iTransformer_Lite, a resource-efficient transformer variant employing simplified embeddings, linear attention, and compact feedforward networks (CompactFFN) to reduce computational overhead; and (2) a multi-class Credit-Based Shaping (CBS) algorithm that leverages iTransformer_Lite predictions for dynamic, SLA-aware traffic shaping. Experiments on multiple public and satellite-specific datasets show that iTransformer_Lite achieves up to approximately 57 % lower memory footprint and approximately 2.3 × faster inference compared to the baseline iTransformer, while maintaining competitive or superior forecasting accuracy across diverse benchmarks.

1. Introduction

Low Earth Orbit (LEO) satellite networks are now recognized as a vital part of the emerging 6G ecosystem, offering truly ubiquitous high-throughput connectivity that reaches remote and underserved areas [1]. Major constellations such as Starlink and Kuiper are deploying hundreds or even thousands of satellites, and these networks must accommodate a wide range of services, including traditional broadband Internet access as well as latency-sensitive applications such as beyond line of sight unmanned vehicle control or immersive real-time video streaming [2]. Although LEO constellations promise global coverage and the potential for very low end-to-end latency, their dynamic topologies, limited onboard compute and memory resources, frequent handovers and massive scale make it challenging to meet stringent service-level agreements (SLAs) across diverse traffic profiles [3,4]. These dynamic conditions not only complicate resource allocation and SLA assurance but also manifest in transient traffic irregularities that further degrade service quality.

Among the various operational challenges stemming from these dynamic topologies, one particularly critical issue is the emergence of microbursts. Frequent orbital movements and the resulting rapid topology changes induce intermittent link quality fluctuations, which in turn give rise to microbursts: transient, high-intensity traffic spikes that can overwhelm limited satellite buffers and lead to packet loss, jitter, and latency spikes. These adverse effects disproportionately impact time-critical applications and thus risk the promise of seamless global coverage. Conventional traffic shaping and time-sensitive networking (TSN) schemes like Cycle Queuing and Forwarding (CQF) and priority based queuing [5–7] are originally designed for relatively stable terrestrial infrastructures and generally lack the agility to anticipate or respond to the brief microburst phenomena characteristic of LEO environments. Although recent onboard solutions address macro-level scheduling and path optimization, they often omit real-time forecasting of traffic surges and tight integration with shaping mechanisms, leaving a critical gap in adaptive traffic management for satellite networks [8,9].

In recent years, researchers have adapted the original Transformer, which excels at capturing long-range dependencies but incurs $O(L^2)$ time and memory costs, to time-series forecasting along four complementary lines. First, **component adaptation** methods such as TiDE [10]

* Corresponding authors at: Shanghai Satellite Network Research Institute Co. Ltd., Shanghai 200120, China.
Email addresses: xuxf@chinasatnet.com.cn (X. Xu), pingdu@ustc.edu (P. Du).

https://doi.org/10.1016/j.neucom.2025.132226
Received 11 August 2025; Received in revised form 4 November 2025; Accepted 26 November 2025
Available online 2 December 2025
0925-2312/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

decompose the signal into trend, seasonal, and residual components for lightweight subnet modeling. FEDformer [11] projects data into the frequency domain and attends only to its low-dimensional spectrum. Autoformer [12] replaces multi-head attention with an Auto-Correlation mechanism to achieve $O(L \log L)$ complexity. Second, **inherent time-series processing** approaches like PatchTST [13] partition each channel into local "patches" (with optional cross-patch attention) to balance local detail and global context, while TimesNet [14] embeds multi-scale convolutions and separable attention into the Transformer backbone, and Stationary/Non-stationary Transformers [15] apply adaptive stationarization and learnable position-encoding kernels to handle shifting data regimes. Third, the **cross-dependency enhancement** [16] paradigm, which is still emerging, updates both components and overall architecture to capture interactions across time and variables simultaneously. Finally, **pure architecture redesign**, as exemplified by iTransformer [17], retains all native components but reconfigures information flow and dimensional interactions to distill knowledge iteratively between encoder and decoder.

Despite the notable accuracy and efficiency improvements achieved by these variants, most still rely on deep stacks or complex attention patterns, making parameter counts, computational demands and memory footprints too large for real time, resource constrained onboard processing in LEO satellite environments. At the same time, although numerous lightweight Transformer architectures have been proposed for vision and NLP such as MobileViT [18] and DistilBERT [19], a purpose-built lightweight transformer model for time series data has yet to appear. Because existing work generally assumes server or cloud based deployment and pays insufficient attention to inference under tight compute and memory budgets, developing and validating a genuinely lightweight time series forecasting model that preserves the iTransformer's ability to model long range dependencies while dramatically reducing parameter count, computational complexity and memory footprint is urgently required for resource limited real time scenarios.

In this paper, we present a lightweight AI-driven traffic forecasting model paired with a dynamic traffic-shaping algorithm, designed for efficient prediction and real-time shaping in compute-constrained environments. We aim to minimize queuing and forwarding delays for high-priority traffic while maintaining compliance with SLAs under variable load conditions. Our main contributions are:

- **iTransformer_Lite: A Resource-Efficient Architecture.** A streamlined Transformer variant for compute-constrained settings, replacing heavy embeddings with a compact projection layer, swapping quadratic self-attention for linear attention, and featuring a bespoke, lightweight feed-forward network, which jointly captures and fuses temporal, spatial, and channel-wise representations—realizing multi-dimensional complementary feature aggregation for accurate yet efficient forecasting.
- **Geotemporal-Aware Dynamic CBS Optimization.** A credit-based traffic-shaping algorithm that leverages S2 geometric partitioning and localized temporal embeddings to integrate iTransformer_Lite's spatiotemporal forecasts into an SLA-driven credit allocation scheme—minimizing delays for high-priority services under fluctuating loads.
- **Comprehensive Evaluation on Diverse Datasets.** As shown in Fig. 1, extensive experiments on public benchmarks and proprietary satellite datasets demonstrate that iTransformer_Lite achieves comparable or superior forecasting accuracy to the state-of-the-art iTransformer while delivering approximately 2.3 × faster inference and up to around 57 % lower memory footprint. Furthermore, in Starlink-based Hypatia simulations, our iTransformer_Lite-base dynamic CBS shaping method achieves an average latency reduction of 3.23 % for high-priority traffic, consistently outperforming the best-performing CBS baseline on each route.

The rest of this paper is organized as follows. Section 2 details the system and traffic models as well as the region-adaptive encoding. Section 3 describes the iTransformer_Lite architecture and its satellite optimizations. Section 4 presents our multi-class CBS algorithms and integration. Section 5 covers experimental setup, datasets, and results. Section 6 concludes with future research directions for adaptive AI-driven traffic management in LEO satellite networks.

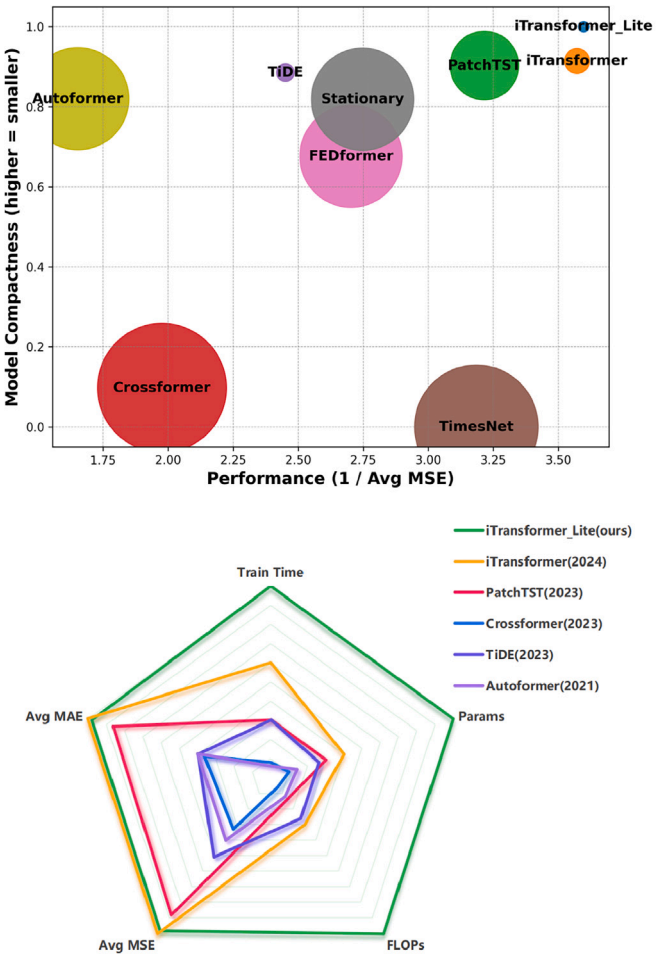


Fig. 1. Trade-off between model compactness and performance.

2. System and traffic models

This section establishes the system model for a LEO satellite network, a domain defined by two fundamental challenges for traffic engineering: (1) high-frequency link quality fluctuations driven by rapid orbital dynamics, and (2) stringent on-board computational constraints that limit the complexity of real-time control algorithms. The models presented herein are specifically formulated to motivate and contextualize the lightweight, predictive framework proposed in this paper. Fig. 2 provides a conceptual overview of this architecture.

2.1. Network dynamics and resource constraints

The LEO constellation is modeled as a time-varying graph $G_t = (V_t, E_t)$, where V_t represents the set of satellite nodes and E_t denotes the set of active inter-satellite links (ISLs). Unlike terrestrial networks, E_t undergoes continuous and predictable changes due to orbital mechanics, with ISL lifetimes often limited to minutes. This dynamic topology introduces significant challenges.

Each satellite node $i \in V_t$ is a resource-constrained compute platform. The processing capacity available for traffic management tasks, such as forecasting and scheduling, is strictly limited. This necessitates that any

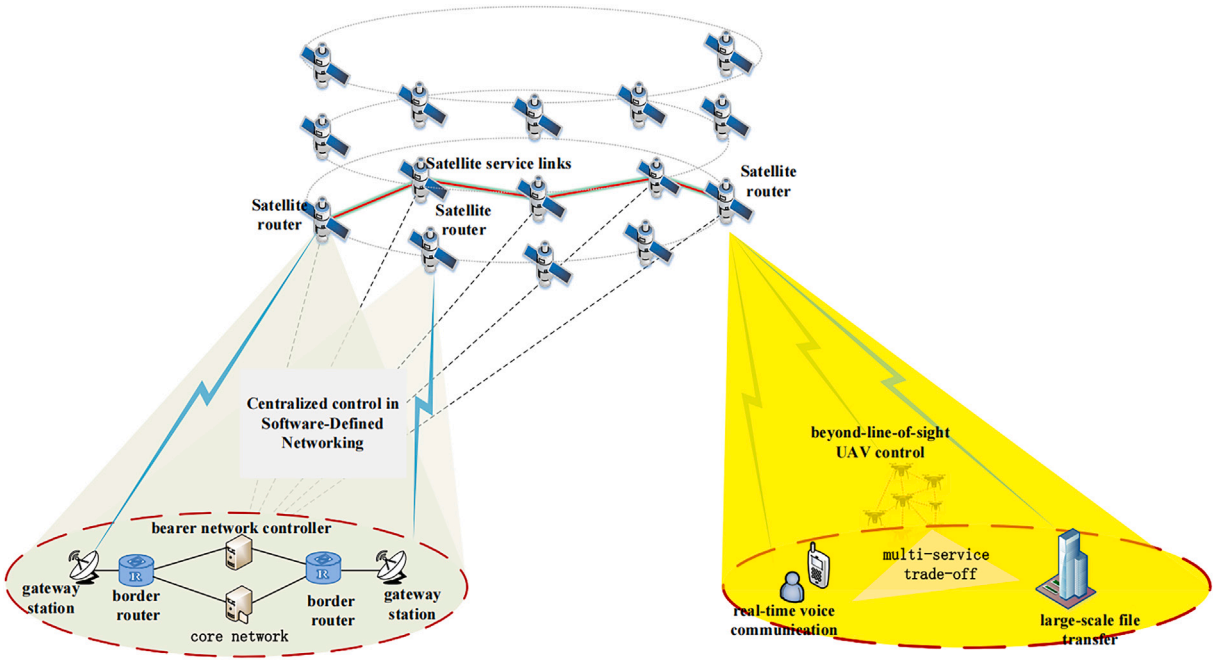


Fig. 2. System architecture for a LEO satellite network. Ground station traffic is uplinked to satellites, where an on-board control plane—equipped with the proposed lightweight forecasting model—dynamically manages inter-satellite link (ISL) resources to ensure end-to-end Quality of Service (QoS).

on-board algorithm must exhibit a computational complexity (measured in FLOPs) and memory footprint that permit execution within a control cycle T_{ctrl} of a few milliseconds.

$$\frac{\text{FLOPs}(f_{\text{forecast}})}{T_{\text{ctrl}}} \leq C_{\text{PU}}^{\max} \tag{1}$$

where FLOPs($f$forecast) is the number of floating-point operations required by the forecasting algorithm f_{forecast}, T_{ctrl} is the duration of the control cycle, and $C_{\text{PU}}^{\max}$ is the maximum processing capacity (in FLOPs per second) of the on-board processing unit.

This fundamental limitation directly motivates our development of a **lightweight forecasting model** to be presented in Section 3, as traditional, computationally expensive models are not viable for this application.

For the laser-based ISLs central to modern constellations, the instantaneous capacity $W_{i,j}(t)$ is highly variable. This variability is not caused by radio-frequency interference, which is negligible in highly directional optical links, but is instead dominated by **pointing errors ($L_p(t)$)**. These errors, arising from minute vibrations and satellite platform jitters, cause rapid fluctuations in the received optical power and, consequently, decrease the link's Signal-to-Noise Ratio (SNR). The effective throughput can be abstractly modeled as a function of this time-varying SNR:

$$W_{i,j}(t) = f(\text{SNR}(t)) \quad \text{where} \quad \text{SNR}(t) \propto \frac{1}{L_p(t)} \tag{2}$$

where $W_{i,j}(t)$ is the instantaneous capacity of the link between satellites i and j, $f(\cdot)$ is a function mapping SNR to throughput (e.g., based on the Shannon-Hartley theorem), and $L_p(t)$ represents the time-varying pointing loss.

When the SNR drops below the threshold required by the Forward Error Correction (FEC) system, the link may experience a temporary outage, leading to severe performance degradation. This reality renders reactive congestion control mechanisms inefficient. Instead, a proactive traffic shaping strategy, driven by forecasts that can anticipate these link state fluctuations, becomes essential for maintaining stable performance. To ensure stability against prediction errors, a safety margin α_{margin} is applied to all capacity allocations [20,21].

2.2. Multi-scale traffic dynamics and microbursts

Traffic originates from ground stations covering distinct geographical regions. The aggregate traffic arrival rate $\lambda_i^m(t)$ at a satellite i for a given class $m \in C$ can be modeled as a composite stochastic process:

$$\lambda_i^m(t) = \bar{\lambda}_i^m + \sum_{k=1}^{K(t)} \xi_k \, \delta(t - \tau_k) + \epsilon_t \tag{3}$$

where $\bar{\lambda}_i^m$ is the baseline rate, and the second term captures intermittent, high-volume microbursts. The heavy-tailed nature of the burst amplitude $\xi_k \sim \text{Pareto}(\alpha)$ [22] reflects that a small fraction of events can dominate short-term traffic volume. These transient surges are capable of overflowing on-board buffers within milliseconds. Consequently, effective mitigation requires a forecasting model with high temporal resolution, capable of anticipating not only the mean load but also the distribution of near-future traffic intensities—this is the primary role of the proposed iTransformer_Lite.

While the microburst phenomenon dominates the sub-second timescale, LEO traffic also exhibits slower variations driven by user activity and regional content demand. To capture this *multi-scale* nature, we further decompose $\lambda_i^m(t)$ into three temporal layers: (1) long-term diurnal variations $\lambda_{\text{day}}(t)$ reflecting global usage cycles, (2) mid-term fluctuations $\lambda_{\text{hour}}(t)$ associated with region-specific events or shifting content popularity, and (3) short-term microbursts $\lambda_{\text{burst}}(t)$ representing high-frequency volatility. These layers interact multiplicatively as:

$$\lambda_i^m(t) = \lambda_{\text{day}}(t) \cdot (1 + \lambda_{\text{hour}}(t)) + \lambda_{\text{burst}}(t) \tag{4}$$

This formulation allows the forecasting model to separate predictable long- and mid-scale patterns from the stochastic microburst component, dedicating higher temporal resolution to the latter.

Physically, microbursts can emerge from synchronization effects in user behavior or backhaul aggregation. For example, simultaneous cloud backups or coordinated telemetry uploads can trigger millisecond-scale packet storms. In LEO constellations, these dynamics are further amplified by the short visibility window of ground-satellite links, causing bursts near handover boundaries or during gateway switching events.

Such cross-layer coupling between traffic demand and orbital dynamics highlights the need for a temporally adaptive predictor whose receptive field aligns with the control period T_{ctrl}.

2.3. Forecast-aware control plane

Building upon this predictive capability, the next step is to integrate traffic forecasts into the network control plane, enabling proactive resource adaptation in response to anticipated load variations. The objective of the on-board Software Defined Network (SDN) controller is to dynamically allocate bandwidth to traffic classes via a Credit-Based Shaper (CBS) [23,24]. At each control interval T_{ctrl}, the controller must solve an optimization problem to adjust the CBS `idleSlope` parameter, denoted by the vector $\mathbf{I}(t) = [I_1(t), \dots, I_C(t)]$, to minimize expected queuing delay while adhering to Service Level Agreement (SLA) constraints.

Let $\hat{\lambda}_i^m(t)$ be the traffic rate for class m at satellite i as predicted by our model, $f_{\text{iTransformer_Lite}}$. The control problem is to find the optimal idle slope allocation $\mathbf{I}^*(t)$:

$$\min_{\mathbf{I}(t)} \quad \mathbb{E}\left[\sum_{m\in C} w_m \cdot D_i^m(t, \mathbf{I}(t))\right] \tag{5}$$

$$\text{s.t.} \quad D_i^m(t) \approx \frac{Q_i^m(t-1) + (\hat{\lambda}_i^m(t) - I_m(t))T_{\text{ctrl}}}{I_m(t)} \tag{6}$$

$$\sum_{m=1}^{C} I_m(t) \le \alpha_{\text{margin}} \cdot W_{i,j}(t), \quad \forall (i,j) \in E_t \tag{7}$$

$$I_m(t) \ge \lambda_m^{\text{SLA}}, \quad \forall m \in C_{\text{protected}} \tag{8}$$

Here, w_m is the priority weight for class m. Eq. (6) provides a first-order approximation of queuing delay, explicitly linking the control action $I_m(t)$ to the predicted arrival rate $\hat{\lambda}_i^m(t)$. This formulation underscores the **critical dependence of the control action on an accurate, real-time traffic forecast**.

2.4. Geotemporal embedding for traffic forecasting

To enable spatiotemporal forecasting in the iTransformer_Lite model, each traffic record is augmented with both location and time features. This joint representation allows the model to learn correlations among geographical regions, local times, and service demand patterns, thus improving forecasting accuracy.

We use Google S2 cells at levels 3–5 (covering approximately 1000–150 km regions) to discretize the Earth's surface. Each traffic sample is assigned to a cell c with a learned embedding $E_{S2}(c) \in \mathbb{R}^{16}$. The corresponding UTC timestamp t_{UTC} is converted into local hour t_{local} using the timezone offset $\Delta_{\text{tz}}(c)$, and encoded cyclically as

$$\phi = \sin\left(2\pi \frac{(t_{\text{UTC}} + \Delta_{\text{tz}}(c)) \bmod 24}{24}\right), \tag{9}$$

$$\psi = \cos\left(2\pi \frac{(t_{\text{UTC}} + \Delta_{\text{tz}}(c)) \bmod 24}{24}\right), \tag{10}$$

where ϕ and ψ denote the two components of the cyclical representation of the local hour, t_{UTC} is the Coordinated Universal Time, and $\Delta_{\text{tz}}(c)$ is the timezone offset of cell c.

The timezone offsets $\Delta_{\text{tz}}(c)$ are stored in a static lookup table indexed by a coarser S2 grid (levels 3 to 5, or an equivalent Beidou partition). Since timezone boundaries are largely stable, this table is generated once on the ground and subsequently uploaded to the satellite as auxiliary metadata. Each grid entry contains a precomputed UTC offset ranging from −12 to +14 hours and, when applicable, a reference to a regional traffic-pattern cluster that characterizes the typical service dynamics of the corresponding area. The global grid consists of approximately 384 to 6144 cells, and each entry occupies three bytes: one byte for the timezone offset and two bytes for the cluster index. The total size of the table is therefore about 2 to 18 kilobytes before compression, a negligible footprint relative to the onboard Flash or ROM capacity. The table can be readily duplicated across all satellites, and updates are required only when revisions occur in the ground-based timezone database.

During inference, only a small subset of cluster profiles is cached in RAM, while the lookup table and compact codebook remain in non-volatile memory. The lookup operation is $O(1)$ and deterministic, keeping runtime memory within a few kilobytes even for multi-day temporal modeling. This design guarantees geographically and temporally consistent predictions across orbital segments, providing a lightweight and autonomous mechanism for region-aware, time-adaptive forecasting without external synchronization.

Each traffic class $m \in C$ is represented as a one-hot vector $\mathbf{b}_m \in \{0,1\}^M$. The final feature vector $\mathbf{v}_{t,c,m}$ for a single sample is the concatenation of spatial, temporal, and class embeddings. The historical input tensor $\mathcal{T}_t \in \mathbb{R}^{k\times R\times D}$ is constructed by stacking the feature matrices from the past k time steps, where R denotes the number of regional series and D the feature dimension.

3. iTransformer_Lite: a lightweight forecasting architecture

To address the stringent onboard computational and control-cycle constraints ($C^{\max}$PU, Tctrl) and the complex network traffic dynamics observed in satellite payload systems (Section 2), conventional forecasting models are impractical. This section introduces **iTransformer_Lite**, a novel architecture specifically engineered for real-time onboard network traffic forecasting under severe computational and memory constraints. We detail its foundational principles and the series of architectural optimizations that drastically reduce computational complexity and memory footprint while preserving high-accuracy spatiotemporal forecasting capabilities. The overall data processing pipeline is illustrated in Fig. 3.

3.1. Foundational principle: inverting the transformer

Traditional Transformers process time series data by attending to the temporal dimension (L). For a multivariate series with N variables, the input is a tensor of shape $[B, L, N]$, where B is the batch size. This approach becomes computationally expensive as the look-back window L increases.

The core innovation of the iTransformer family, which we adapt, is to **invert this perspective**. We treat the individual time series (variables) as tokens, not the time steps. The input tensor is transposed and projected to $[B, N, d_{\text{model}}]$, where each of the N variables is represented by a feature vector of length d_{model} that encodes its behavior over the entire time window L.

$$\text{Input} \in \mathbb{R}^{B\times L\times N} \xrightarrow{\text{Projection}} \text{Tokens} \in \mathbb{R}^{B\times N\times d_{\text{model}}} \tag{11}$$

where B is the batch size, L is the length of the look-back window, N is the number of variables (time series), and d_{model} is the model's embedding dimension.

This allows the self-attention mechanism to model the interrelationships between the N different traffic flows directly, making the learned dependencies between variables explicitly interpretable as inter-regional correlations that drive future traffic dynamics. In satellite networks, this formulation effectively captures the correlations among geographically or topologically adjacent beams or regions, enabling more accurate spatiotemporal traffic forecasting.

3.2. Instance normalization for stability

To handle the non-stationary nature of traffic data, where distributions shift over time, we employ an instance normalization technique inspired by RevIN [25]. Before entering the encoder, each time series instance in the batch is normalized by its own mean and standard deviation. After the forecast is generated, the output is denormalized using

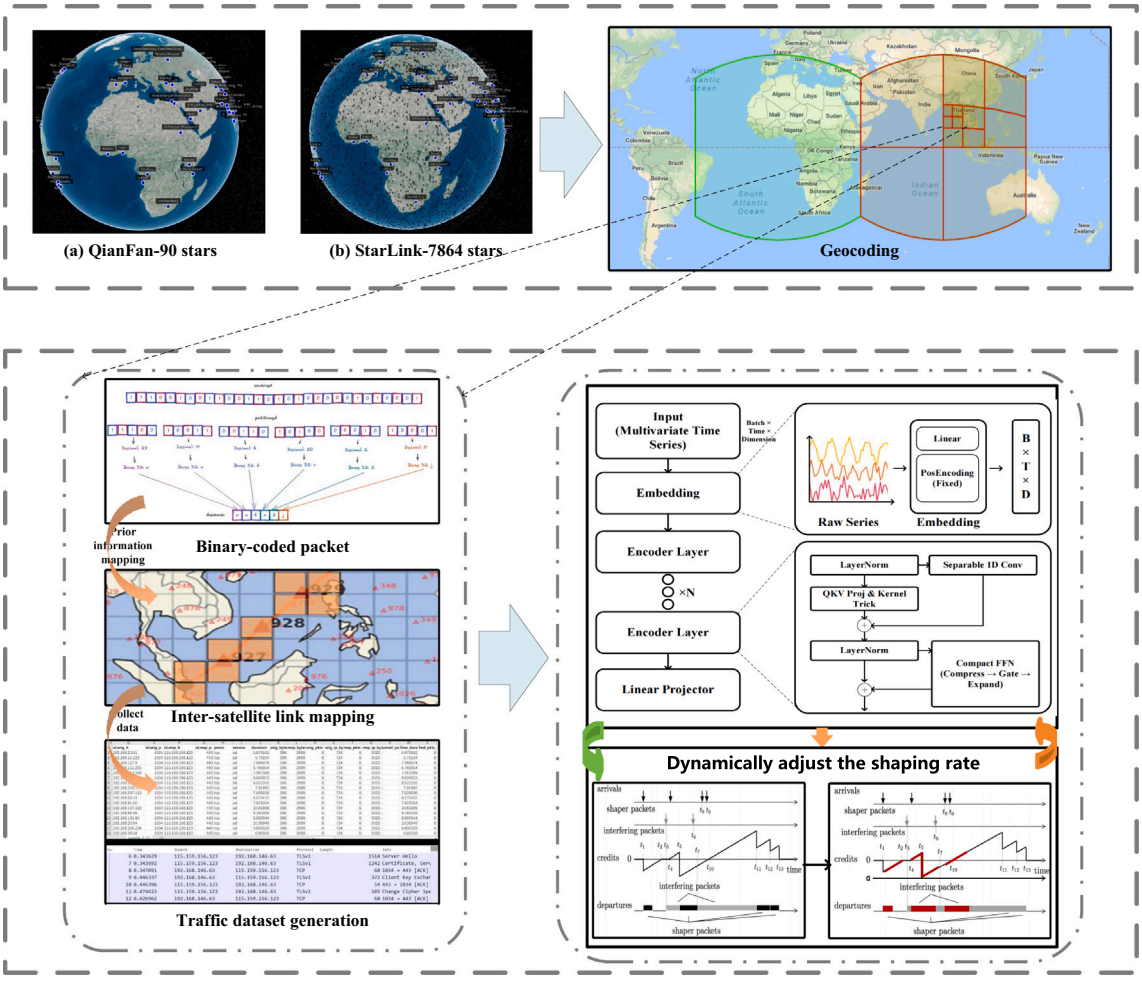


Fig. 3. Pipeline for geocoded regional traffic data collection, iTransformer_Lite forecasting, and CBS-based dynamic traffic shaping.

the stored statistics.

$$\mathbf{x}' = \frac{\mathbf{x} - \mu(\mathbf{x})}{\sigma(\mathbf{x})} \tag{12}$$

$$\hat{\mathbf{y}} = \hat{\mathbf{y}}'\sigma(\mathbf{x}) + \mu(\mathbf{x}) \tag{13}$$

where $\mathbf{x}$ is the input time series instance, $\mathbf{x}'$ is its normalized version, $\mu(\mathbf{x})$ and $\sigma(\mathbf{x})$ are the mean and standard deviation of the instance, $\hat{\mathbf{y}}'$ is the model's raw (normalized) forecast, and $\hat{\mathbf{y}}$ is the final denormalized forecast.

This simple yet effective step stabilizes the training process and improves generalization across different traffic patterns, as it enables the model to focus on relative temporal variations rather than absolute magnitudes, enhancing interpretability under shifting traffic distributions.

3.3. Architectural components and optimizations

The "Lite" architecture is encoder-only and systematically optimizes each component for a minimal computational footprint.

3.3.1. Efficient embedding layer

We replace complex embeddings with a more direct approach. The input series (length L) is projected into the model dimension d_{model} using a single linear layer. Instead of fixed sinusoidal encodings, we add a simple, **learnable embedding vector** to supply positional information.

$$\mathbf{X}_{\text{emb}} = \text{Linear}(\mathbf{X}_{\text{permuted}}) + \mathbf{E}_{\text{pos}} \tag{14}$$

where $\mathbf{X}_{\text{permuted}} \in \mathbb{R}^{B\times N\times L}$ and the learnable embedding $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{1\times 1\times d_{\text{model}}}$ is broadcast across the batch and variable dimensions. This is both parameter-efficient and adaptive, preserving positional awareness while allowing each variable's embedding to reflect its long-term temporal trend, which improves interpretability of the input representation.

3.3.2. Efficient linear attention

The standard $O(N^2)$ self-attention is replaced with a **kernelized linear attention** of $O(N)$ complexity. We use the ELU activation function as a kernel $\phi(\cdot)$ to ensure positivity. The computation is defined as:

$$\text{LinAtt}(Q,K,V) = \frac{\phi(Q)\left(\phi(K)^\top V\right)}{\phi(Q)(\phi(K)^\top \mathbf{1})} \tag{15}$$

$$\phi(x) = \text{elu}(x) + 1 \tag{16}$$

where Q, K, V are respectively the query, key, and value matrices derived from the input. $\phi(\cdot)$ is the kernel function ensuring positivity, and $\mathbf{1}$ is a vector of ones used for normalization.

The denominator term provides an explicit normalization for each query, enhancing stability without causing significant computational overhead, and making the resulting attention weights directly interpretable as normalized measures of inter-variable influence.

3.3.3. Sequential hybrid encoder layer

Our encoder layer adopts a resource-efficient sequential structure that fuses local and global features. An input $\mathbf{X}$ first passes through a linear attention and a separable convolution layer simultaneously. Their

outputs are added to the original input via a residual connection. This aggregated feature set is then passed to a compact feed-forward network (CFFN).

$$\begin{aligned}\mathbf{H} &= \text{LayerNorm}(\mathbf{X} + \text{LinAtt}(\mathbf{X}) + \text{SepConv1D}(\mathbf{X}))\\ \mathbf{Y} &= \text{CFFN}(\mathbf{H})\end{aligned} \tag{17}$$

where $\mathbf{X}$ is the input to the encoder layer, $\mathbf{H}$ is the intermediate representation fusing global (LinAtt) and local (SepConv1D) features, and $\mathbf{Y}$ is the final output of the layer after the CFFN.

This design ensures that the non-linear transformation in the FFN acts upon a rich representation containing both global (attention) and local (convolution) information, clarifying how global dependencies and localized temporal fluctuations jointly contribute to the final prediction.

3.3.4. Bottleneck separable convolution

Our 1D separable convolution, 'SepConv1D', is implemented with a bottleneck structure to further reduce parameters. After the initial depthwise convolution, the features are projected down to a compressed dimension by a pointwise convolution, passed through a GELU activation, and then projected back to the original dimension.

3.3.5. Compressed feed-forward network (CFFN)

The standard feed-forward network (FFN) is replaced with a compressed variant employing a SwiGLU gating mechanism, which provides better performance with fewer parameters than a standard ReLU-based FFN.

$$\text{CFFN}(\mathbf{X}) = (\text{SiLU}(\mathbf{X}W_{\text{gate}}) \odot (\mathbf{X}W_{\text{up}}))W_{\text{down}} \tag{18}$$

where $\mathbf{X}$ is the input tensor, W_{gate}, W_{up}, and W_{down} are the learnable weight matrices of the linear layers, SiLU is the Sigmoid Linear Unit activation function, and $\odot$ denotes element-wise multiplication.

This gated structure allows the network to dynamically control the information flow, improving expressiveness. Unlike the conventional Transformer FFN, which expands the hidden dimension by a factor of 4 and then projects it back, the CFFN compresses the intermediate representation to a lower dimension $d_{\text{mid}} < d_{\text{model}}$ before applying the gating operation. This design reduces both the parameter count and computational cost by approximately 30 %–40 % without sacrificing accuracy. The gating branch $\text{SiLU}(\mathbf{X}W_{\text{gate}})$ adaptively scales the activation of $\mathbf{X}W_{\text{up}}$, enabling the model to retain salient features while suppressing redundant components, thus providing an interpretable mechanism for feature importance, where higher gate activations indicate more influential latent dimensions in the forecasting process. To further enhance efficiency, all CFFN layers share a quantized projection basis for W_{up} and W_{down}, which minimizes memory transfers and facilitates on-board inference on low-power processors.

These lightweight strategies jointly form a coherent and interdependent design, where each component contributes to computational efficiency while preserving essential representational capacity. Through this coordinated architecture, global dependencies, local temporal dynamics, and channel-wise interactions are effectively maintained, ensuring that parameter reduction does not compromise the model's ability to capture critical information.

4. Forecast-driven CBS for proactive traffic shaping

Having established the efficiency and accuracy of iTransformer_Lite, this section details its practical application: a **prediction-driven, multi-class Credit-Based Shaper (CBS)** algorithm. Standard CBS mechanisms are inherently *reactive*, which means that they adjust to traffic congestion only after it has occurred. This reactive nature is a significant drawback in highly dynamic LEO satellite networks. Our proposed method fundamentally shifts this paradigm from reactive to *proactive* by integrating the traffic forecasts from iTransformer_Lite. By anticipating near-future traffic loads, the system can pre-emptively adjust bandwidth allocations, ensuring high-priority services maintain stringent SLA guarantees with minimal latency, even under the volatile conditions of satellite links.

4.1. Methodology: from traffic forecasts to dynamic rate allocation

The standard multi-class CBS partitions outgoing traffic into C classes, each regulated by an *idle slope* I_c (credit accumulation rate) and a *send slope* S_c (credit consumption rate). The credit for class c, denoted $K_c(t)$, evolves according to:

$$K_c(t) = \begin{cases} I_c, & \text{if class } c \text{ is idle} \\ -S_c, & \text{if class } c \text{ is transmitting} \end{cases} \tag{19}$$

where a frame can only be transmitted if its class has non-negative credit ($K_c(t) \geq 0$).

Our core innovation is to make this process forward-looking. We leverage the forecast $\hat{\lambda}_c(t+\Delta)$ for the next time interval, provided by iTransformer_Lite, to compute a normalized, predictive weight $\alpha_c(t)$ for each class:

$$\alpha_c(t) = \frac{\hat{\lambda}_c(t+\Delta)}{\sum_{j=1}^{C} \hat{\lambda}_j(t+\Delta)}, \quad \text{where} \quad \sum_{c=1}^{C} \alpha_c(t) = 1 \tag{20}$$

This weight represents the anticipated share of traffic for class c. We use it to define a *target idle slope* $I_c^*(t)$ that reflects this predicted demand:

$$I_c^*(t) = I_{\text{tot}} \cdot \alpha_c(t), \quad \text{such that} \quad \sum_{c=1}^{C} I_c^*(t) = I_{\text{tot}} \tag{21}$$

where I_{tot} is the total shaping rate of the physical link. This target rate serves as an ideal allocation based on future, rather than past network state.

4.2. The forecast-driven CBS algorithm

At each control epoch $t_k = k\Delta$, the on-board controller executes Algorithm 1 to update the shaping parameters. This algorithm is designed to be both responsive to predictions and robust against forecast noise, while rigorously enforcing service guarantees.

The proposed algorithm is designed to overcome the limitations of the simpler baselines discussed in Section 5. It integrates forecasting, smoothing, and capacity management into a unified predictive control framework, guided by the pseudocode presented in Algorithm 1.

Unlike reactive schemes such as *PlainCBS* or rule-based variants relying on historical averages, the method exploits high-fidelity forecasts from *iTransformer_Lite* to anticipate and mitigate traffic surges in advance. This proactive design allows the system to prepare rather than respond, reducing oscillations and improving latency stability.

The smoothing coefficient γ acts as an adaptive regulator that balances responsiveness against noise sensitivity, ensuring steady operation even under transient prediction errors. Bandwidth allocation then proceeds according to the forecast-informed control logic: each service class is guaranteed its minimum SLA bandwidth λ_c^{SLA}, and if the total demand exceeds the link capacity I_{tot}, only the non-protected classes are proportionally scaled down while preserving contractual guarantees.

Through this synthesis of accurate lightweight forecasting and disciplined resource allocation, the predictive-CBS scheme achieves both fairness and efficiency. The following section provides empirical validation, demonstrating clear gains in latency and stability across high-fidelity simulations.

5. Experiments and results

In this section, we describe the experimental setup and present the results for two validation tasks: (1) forecasting performance of *iTransformer_Lite* on seven public real-world datasets and an additional "LEO Traffic" dataset, comparing both accuracy and computational efficiency against state-of-the-art baselines; (2) traffic shaping (CBS) performance using *iTransformer_Lite*-based dynamic CBS versus conventional CBS and fixed-rule CBS, which were evaluated on historical traffic data.

Algorithm 1 Forecast-Driven Multi-Class CBS Shaping.

Require: Previous idle slopes $\{I_c(t_{k-1})\}$, smoothing factor $\gamma \in (0, 1]$, SLA rates $\{\lambda_c^{\mathrm{SLA}}\}$.
Ensure: Updated idle slopes $\{I_c(t_k)\}$.
Forecast: Query iTransformer_Lite for predicted rates $\hat{\lambda}_c(t_k + \Delta)$ for all classes $c \in \{1, \ldots, C\}$.
Weight Computation: Compute predictive weights $\alpha_c \leftarrow \hat{\lambda}_c / \sum_j \hat{\lambda}_j$.
Target Rate Calculation: Determine target idle slopes $I_c^* \leftarrow I_{\mathrm{tot}} \cdot \alpha_c$.
Smoothed Update: Apply exponential smoothing to prevent oscillations and filter noise from forecasts:
$I_c' \leftarrow (1 - \gamma) \cdot I_c(t_{k-1}) + \gamma \cdot I_c^*$.
SLA Enforcement: Guarantee minimum bandwidth by enforcing the SLA floor for each class:
$I_c(t_k) \leftarrow \max(I_c', \lambda_c^{\mathrm{SLA}})$.
Capacity Normalization: If total allocated bandwidth exceeds link capacity, proportionally scale down all non-protected classes to ensure the sum equals I_{tot}.
if $\sum_c I_c(t_k) > I_{\mathrm{tot}}$ **then**
  $I_{\mathrm{over}} \leftarrow \sum_c I_c(t_k) - I_{\mathrm{tot}}$.
  $I_{\mathrm{scalable}} \leftarrow \sum_{j \text{ s.t. } I_j > \lambda_j^{\mathrm{SLA}}} (I_j - \lambda_j^{\mathrm{SLA}})$.
  for each class c where $I_c(t_k) > \lambda_c^{\mathrm{SLA}}$ **do**
    $I_c(t_k) \leftarrow I_c(t_k) - I_{\mathrm{over}} \cdot \frac{I_c(t_k) - \lambda_c^{\mathrm{SLA}}}{I_{\mathrm{scalable}}}$.
  end for
end if
Apply Update: Use the finalized set of idle slopes $\{I_c(t_k)\}$ in the CBS hardware for the next interval.

5.1. Forecasting performance evaluation

5.1.1. Datasets

We evaluate *iTransformer_Lite* on seven widely used real-world time series datasets. The ETT dataset [12] comprises four subsets—ETTh1 and ETTh2 are recorded hourly, and ETTm1 and ETTm2 are recorded every 15 min—spanning July 2016 to July 2018. The Exchange dataset [12] contains daily foreign-exchange rates for eight countries from 1990 to 2016. The Weather dataset [12] includes 21 meteorological factors sampled every 10 min at the Max Planck Biogeochemistry Institute in 2020. The ECL dataset [12] records hourly electricity consumption for 321 clients. The Traffic dataset [12] measures hourly road occupancy rates from 862 sensors in the San Francisco Bay Area from January 2015 to December 2016. The Solar-Energy series [26] captures 10-minute solar power output from 137 PV plants throughout 2006. The PEMS dataset [27] comprises four 5-minute-resolution subsets (PEMS03, PEMS04, PEMS07, PEMS08) of California traffic network data.

5.1.2. Baselines

We evaluate *iTransformer_Lite* against a range of recent forecasting methods, including its full-sized counterpart **iTransformer** [28], lightweight linear and attention-based variants such as **RLinear** [29], **PatchTST** [13], **Crossformer** [16], **TiDE** [10], **TimesNet** [14], and **DLinear** [30]. We also include classical and foundational models like **SCINet** [27], **FEDformer** [11], **Stationary** [15], and **Autoformer** [12]. Wherever available, we follow the original data splits, input/output horizon settings, and evaluation metrics reported in the respective works to ensure a fair and reproducible comparison.

5.1.3. Experimental setup

All experiments were implemented in PyTorch and conducted on a single NVIDIA RTX 2060 GPU (12 GB memory) paired with an Intel Core i3 CPU. The training protocol followed standard practices used in comparable works [13,16,28]: all models were trained for an equal number of epochs using the AdamW optimizer with identical learning rate schedules, batch sizes, and early stopping criteria. When baseline methods incorporated data augmentation or specific regularization strategies, their original configurations were strictly preserved to ensure a fair comparison.

To further evaluate real-world deployability, we benchmarked both *iTransformer* and *iTransformer_Lite* under a **resource-constrained satellite environment**. Specifically, inference speed, training latency, and memory usage were tested on a single-core ARM Cortex-A72 CPU (Raspberry Pi 4 platform), simulating embedded onboard processors used in low Earth orbit (LEO) satellites. The evaluation was conducted under single-thread execution with a batch size of 1, input sequence length of 96, and prediction length of 96 to emulate actual onboard inference conditions.

5.1.4. Results and analysis

This section evaluates iTransformer_Lite in terms of computational efficiency and forecasting accuracy. Results show that the lightweight model greatly reduces computation while maintaining comparable or even better predictive performance than the original iTransformer.

Computational efficiency was analyzed via parameter count (#Params), floating-point operations (FLOPs), and training time per epoch.

To further validate practical efficiency, both models were benchmarked on a single-core ARM Cortex-A72 CPU (Raspberry Pi 4), representing a low Earth orbit (LEO) onboard processor. Experiments used single-thread execution with batch size = 1 and sequence/prediction length = 96 to emulate onboard inference.

Table 1 shows that iTransformer_Lite achieves up to 45 % lower FLOPs and 18 % fewer parameters, leading to reduced memory usage and faster inference.

On the Traffic dataset, mean latency decreases from 439.0 ms to 187.7 ms (>2 × speed-up), and peak memory drops by about 17 %, confirming suitability for **resource-limited onboard environments**. For the lighter Weather task, latency remains nearly unchanged while memory usage slightly decreases.

To further validate the design choices in iTransformer_Lite, we conducted an ablation study to analyze the contributions of each lightweight component: Linear Attention, SepConv1D, and CFFN. The experiments were performed on the **Weather** dataset [12], which records 21 meteorological variables every 10 min at the Max Planck Biogeochemistry Institute (2020). This dataset was selected due to the strong overall performance of iTransformer_Lite on it.

We evaluated variants of iTransformer_Lite by selectively removing each of the three components while keeping other configurations identical. Forecasting horizons of 96, 192, 336, and 720 were tested, and we report the average MAE and MSE across all horizons to highlight overall trends.

Table 1
Computational complexity, memory footprint (RSS), and inference latency on ARM Cortex-A72 CPU.

Dataset	Model	#Params (M)	FLOPs/Sample (G)	RSS Base (MB)	RSS Peak (MB)	Mean Latency (ms)	p95 (ms)
Traffic	iTransformer	6.41	8.63	324.85	345.40	439.0	493.2
	iTransformer_Lite	5.23	4.75	287.10	287.12	187.7	209.7
Weather	iTransformer	4.83	0.12	270.60	270.67	9.1	11.9
	iTransformer_Lite	3.95	0.09	266.28	266.30	9.1	10.5

Table 2
Ablation results on the weather dataset (average across forecast lengths).

Model	Avg. MAE	Avg. MSE
iTransformer	0.258	0.278
iTransformer_Lite	0.251	0.275
iTransformer_Lite (w/o SepConv1D)	0.259	0.279
iTransformer_Lite (w/o Linear Attention)	0.256	0.278
iTransformer_Lite (w/o CFFN)	0.253	0.276

As shown in Table 2, removing any individual component leads to a measurable degradation in forecasting performance, confirming that each module contributes positively and complements the others. Specifically, **SepConv1D** improves local temporal feature extraction, **Linear Attention** lowers computational complexity while maintaining long-range dependencies, and **CFFN** enhances non-linear representation capability. Together, these mechanisms yield a synergistic improvement, analogous to multi-path feature extraction in ResNet-like architectures, enabling higher efficiency and accuracy without significantly increasing computational cost.

Table 3 provides a comprehensive comparison of forecasting accuracy (MSE and MAE) across seven benchmark datasets. The results highlight the remarkable performance of **iTransformer_Lite**, which not only competes with but often surpasses its larger counterpart and other leading models. For **iTransformer_Lite**, each reported value represents the mean result over five independent runs. The corresponding standard deviations are very small (no greater than 0.004 for either MSE or MAE), confirming the high stability and reproducibility of the model's performance. For all other baselines, we use the benchmark results reported in the iTransformer (2024) paper to ensure consistency in comparison.

An analysis of the results reveals a compelling pattern. On the ECL, Exchange, and Weather datasets, iTransformer_Lite achieves state-of-the-art performance, outperforming all other models—including the original iTransformer. For instance, on the ECL dataset, it attains an MSE/MAE of 0.172/0.266, surpassing iTransformer's 0.178/0.270. On the Traffic, Solar-Energy, and PEMS (Avg) datasets, iTransformer_Lite ranks as a close runner-up, trailing only slightly behind the full iTransformer model. Even in these cases, the performance gap remains minimal, and the differences lie within the standard deviation range, indicating that the two models perform comparably under normal experimental variations.

Overall, across all seven datasets, iTransformer_Lite consistently places either first or second relative to its predecessor, demonstrating that its lightweight design does not compromise accuracy. In fact, its superior results on three datasets suggest that the streamlined architecture may enhance generalization and mitigate overfitting in certain contexts.

In summary, **iTransformer_Lite** provides a superior trade-off between computational cost and predictive accuracy. It establishes a new benchmark for efficiency while delivering forecasting performance that is on par with, or even better than, more complex and computationally expensive models. This makes it an outstanding choice for practical, real-world time-series forecasting applications with resource constraints, while maintaining statistically validated improvements and strong stability across repeated experiments.

5.2. Performance evaluation of forecast-driven traffic shaping

This section presents a comprehensive evaluation of our proposed traffic shaping framework, which integrates the lightweight **iTransformer_Lite** forecasting model with the dynamic, proactive CBS detailed in Section 4. The entire system is designed to address the fundamental challenges of LEO satellite networks: highly dynamic topologies, variable link quality, and stringent on-board computational constraints, as established in our system model (Section 2). We empirically validate that our proactive, forecast-driven method outperforms traditional reactive and rule-based methods in this demanding environment.

5.2.1. Experimental setup

All experiments were conducted using the Hypatia platform [31], a high-fidelity simulator purpose-built for modeling LEO satellite constellations. The platform accurately models satellite mobility and dynamic inter-satellite link (ISL) behavior. To account for the physical-layer instability caused by pointing errors as discussed earlier, the nominal capacity of all ISLs is modeled with a conservative safety margin (α_{margin}) applied. To ensure our results reflect real-world conditions, the simulation was configured using the **SpaceX Starlink constellation** parameters (550 km altitude, 72 satellites per 22 orbital planes, 53° inclination). The end-to-end delay was meticulously calculated, accounting for ground-satellite propagation, multi-hop ISL propagation, handover switching delays, and on-board processing overhead.

The experimental workflow, illustrated in Fig. 4, begins with the generation of multi-class traffic flows that emulate nine distinct real-world service types. The integrated forecasting model is then applied to predict the near-future traffic rates for each service class. Based on these forecasts, the CBS idle slope parameters are dynamically adjusted following the procedure outlined in Algorithm 1. The shaped traffic is subsequently mapped onto the simulated LEO network's bearer resources, and key performance indicators (KPIs)—primarily end-to-end latency—are recorded for evaluation.

We evaluate our proposed iTransformer_Lite-driven CBS against four representative baseline algorithms to ensure a comprehensive comparison. The Prediction-Driven CBS employs a hybrid forecasting scheme that integrates a linear trend fit [32], an autoregressive model with up to four lags [33], and a simple seasonal average, fused through a class-specific `prediction_weight`. It pre-reserves bandwidth proportional to prediction confidence and applies overflow penalties, priority and buffer-utilization adjustments, uncertainty penalties, and burst-tolerance checks to derive final latency. The

Table 3
Forecasting results (MSE, MAE) on various datasets. Red indicates the best performance, and blue indicates the second-best.

Model	ECL		ETT (Avg)		Exchange		Traffic		Weather		Solar-Energy		PEMS (Avg)	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
iTransformer_Lite (ours)	0.172	0.266	0.382	0.399	0.354	0.402	0.430	0.284	0.251	0.275	0.238	0.269	0.120	0.225
iTransformer (2024)	0.178	0.270	0.383	0.399	0.360	0.403	0.428	0.282	0.258	0.278	0.233	0.262	0.119	0.218
RLinear (2023)	0.219	0.298	0.380	0.392	0.378	0.417	0.626	0.378	0.272	0.291	0.369	0.356	0.514	0.482
PatchTST (2023)	0.205	0.290	0.381	0.397	0.367	0.404	0.481	0.304	0.259	0.281	0.270	0.307	0.217	0.305
Crossformer (2023)	0.244	0.334	0.685	0.578	0.940	0.707	0.550	0.304	0.259	0.315	0.641	0.639	0.220	0.304
TiDE (2023)	0.251	0.344	0.482	0.470	0.370	0.413	0.760	0.473	0.271	0.320	0.347	0.417	0.375	0.440
TimesNet (2023)	0.192	0.295	0.391	0.404	0.416	0.443	0.620	0.336	0.259	0.287	0.301	0.319	0.148	0.246
DLinear (2023)	0.212	0.300	0.442	0.444	0.354	0.414	0.625	0.383	0.265	0.317	0.330	0.401	0.320	0.394
SCINet (2022)	0.268	0.365	0.689	0.597	0.750	0.626	0.804	0.509	0.292	0.363	0.282	0.375	0.121	0.222
FEDformer (2022)	0.214	0.327	0.408	0.428	0.519	0.429	0.610	0.376	0.309	0.360	0.291	0.381	0.224	0.327
Stationary (2022)	0.193	0.296	0.471	0.464	0.461	0.454	0.624	0.340	0.288	0.314	0.261	0.381	0.151	0.249
Autoformer (2022)	0.227	0.338	0.465	0.459	0.613	0.539	0.628	0.379	0.338	0.382	0.885	0.711	0.614	0.575

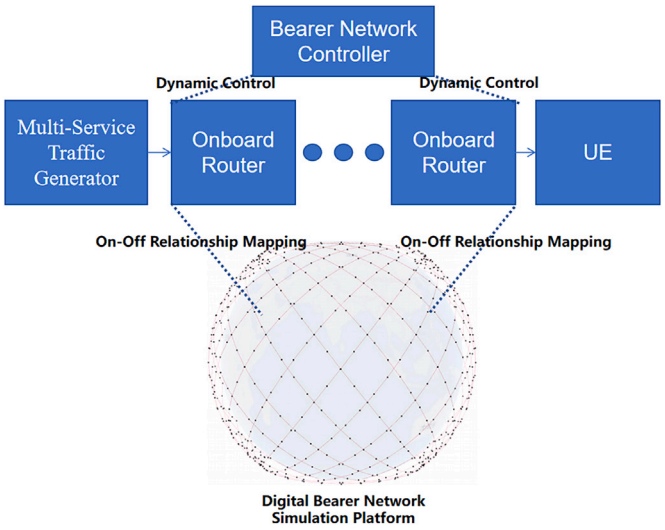


Fig. 4. Hypatia-based simulation framework for evaluating forecast-driven CBS.

TimeAwareRuleCBS uses an EWMA-based forecast [34] within a sliding window and dynamically adapts per-class bandwidth shares by comparing short-term and window-wide averages (growth/decline thresholds of 1.2/0.8), subsequently enforcing latency caps through CBS scheduling.

For comparison with simpler heuristics, PlainCBS represents the standard reactive implementation that relies solely on the most recent traffic sample as a naive forecast, allocating static bandwidth shares and computing latency from queue and buffer conditions. The FixedRuleCBS baseline, by contrast, applies a five-point moving average of past traffic together with fixed per-class latency and sensitivity coefficients to linearly estimate final delay.

5.2.2. Forecasting accuracy on LEO traffic

The efficacy of our proactive control plane is critically dependent on the accuracy and efficiency of its underlying traffic forecasts. We first validated the performance of **iTransformer_Lite** on a specialized multi-class **LEO Traffic** dataset, covering nine service categories. Table 4 compares its forecasting accuracy (MSE and MAE) against other state-of-the-art time-series models.

The **LEO Traffic** dataset is not derived from direct satellite telemetry but is instead synthesized through a high-fidelity mapping of anonymized terrestrial network traces to emulate low Earth orbit (LEO) communication traffic patterns. Specifically, we utilized desensitized ground network traces across multiple service categories—including control, video streaming, synchronization, and IoT applications—and assigned each service a corresponding *priority class* and *latency constraint* following representative LEO mission profiles. The dataset spans one continuous week of traffic sampled every 10 s (10,080 points in total), containing fields such as *timestamp*, *service type*, *priority level*, *traffic load (Mbps)*, *latency (ms)*, and *packet loss rate*.

It is important to note that dataset generation is *decoupled* from Hypatia itself. Hypatia functions as a dynamic orbital network simulation engine, which uses orbital geometry and inter-satellite link (ISL) dynamics to generate time-varying routing paths and hop counts. Based on these simulated routes, Hypatia estimates per-service forwarding and propagation latency across time slices. The **LEO Traffic** dataset then acts as the input traffic workload to Hypatia, enabling end-to-end evaluation of communication performance under realistic, dynamically changing link conditions.

The results unequivocally demonstrate that **iTransformer_Lite** achieves the best average forecasting performance, with an overall MSE of **0.9969** and MAE of **0.2864**. Crucially, it not only outperforms

Table 4
Forecasting accuracy (MSE/MAE) by service category on simulated LEO traffic. Our lightweight model achieves the best overall performance.

Model	FileTransfer		VoIP		IoTControl		VideoStream		Gaming		DataSync		WebBrowse		VideoConf		Email		**Average**	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Autoformer	1.4748	0.3822	1.0219	0.4048	0.7818	0.2473	1.2474	0.4597	1.7325	0.6384	0.6530	0.4506	1.6938	0.4335	6.2820	1.9748	1.2501	0.5083	1.6819	0.5666
PatchTsT	1.4932	0.3155	1.1424	0.3615	0.4991	0.2355	1.4091	0.5946	1.0303	0.2182	0.5943	0.4219	1.5888	0.3716	0.3282	0.1949	1.1860	0.4369	1.0302	0.3501
iTransformer	1.4820	0.3166	1.0120	0.2249	0.4011	0.2046	1.3365	0.3217	1.0254	0.2091	0.6563	0.4820	1.5667	0.3372	0.3275	0.1317	1.2380	0.3806	1.0051	0.2898
iTransformer_Lite (ours)	1.4801	0.3199	1.0231	0.2328	**0.3275**	**0.1981**	**1.2776**	**0.2849**	**1.0375**	**0.2055**	0.7018	0.4780	1.6116	0.3518	0.3979	0.1660	**1.1148**	**0.3405**	**0.9969**	**0.2864**

Table 5
Comparison of on-board processing and queuing delay (ms).

Algorithm	Mean	Std Dev	Stability	Range
iTransformer_Lite	**0.131**	**0.007**	**1.000**	[0.028, 0.242]
Prediction-Driven CBS	0.448	0.075	0.990	[0.055, 1.000]
TimeAwareRuleCBS	0.184	0.005	1.000	[0.050, 0.310]
PlainCBS	0.882	1.098	0.111	[0.070, 8.258]
FixedRuleCBS	3.753	1.864	0.034	[0.237, 9.600]

its larger counterpart (iTransformer) on average but also shows superior accuracy for critical, latency-sensitive services like `IoTControl`, `VideoStream`, and `Gaming`.

This empirical advantage further reinforces that our lightweight architecture can deliver precise, low-latency predictions for on-board control and scheduling within LEO networks, under realistic traffic patterns generated via Hypatia's orbital dynamics engine.

A critical component of the total end-to-end latency in a LEO network is the delay incurred within each satellite's on-board router, which includes processing overhead and queuing time. The ultimate success of our framework hinges on its ability to minimize this nodal delay through superior forecasting, thereby contributing to a tangible reduction in the overall end-to-end network latency. We evaluated all five algorithms to quantify their impact on this specific delay component.

Table 5 summarizes the on-board processing and queueing delay statistics aggregated across all test scenarios. Our **iTransformer_Lite**-driven approach excels, achieving a mean on-board delay of just **0.131 ms**. This represents a **70.8 %** reduction in this specific delay component compared to the next-best baseline (TimeAwareRuleCBS) and a reduction of over **96 %** compared to simple reactive baselines. This starkly illustrates how minimizing per-hop processing and queueing overhead through proactive shaping is a key enabler for high-performance communication. The reduction of this nodal delay is a primary contributor to the improved end-to-end latency across city pairs, as will be detailed in the subsequent section.

To assess the real-world impact of our framework, we analyzed the latency of the satellite **transport network segment**. This metric represents the delay between ground gateways in different cities and is a crucial indicator of the core network's performance. The transport network latency comprehensively includes: 1) the multi-hop on-board processing and queuing delays, 2) the propagation time across inter-satellite links (ISLs), 3) the average overhead from the transport link protocol, and 4) errors arising from laser link pointing and signal loss. Notably, this measurement deliberately excludes the access network segment (i.e., the first- and last-hop user-to-satellite links) to specifically isolate and evaluate the performance of the satellite backbone itself.

Figs. 5 through 9 visualize the transport latency over time for each city pair, while Table 6 provides a quantitative summary.

As summarized in Table 6, our **iTransformer_Lite**-driven method consistently records the lowest **transport network latency** across all five distinct routes. The performance improvement over the most sophisticated rule-based baseline (TimeAwareRuleCBS) ranges from 4.3 % to 4.4 % on various routes. However, compared to the simplistic FixedRuleCBS, the latency reduction is dramatic, ranging from **14.4 %** to a remarkable **38.8 %** (on the Shanghai-Beijing route). This confirms that proactive traffic shaping, powered by accurate and lightweight forecasting, is a key enabler for high-performance communication across the next-generation LEO satellite transport network.

Although the numerical latency reduction between **iTransformer_Lite** and **TimeAwareRuleCBS** (4 %) may appear modest, it is *practically meaningful* under realistic low-Earth-orbit (LEO) service-level agreements (SLAs). In multi-hop satellite paths, end-to-end latency accumulates across successive inter-satellite and inter-orbit hops, each typically contributing 2–5 ms of delay. This value derives from the physical propagation distance of 700–1500 km per hop at the speed of light ($c \approx 3.0 \times 10^8$ m/s), yielding 2.3–5.0 ms propagation delay, plus minimal switching and queuing overhead. Consequently, even a 4 % reduction in total latency can translate into an 8–12 ms improvement over long intercontinental routes—e.g., Paris-Sydney—where multiple hops are involved.

Such an improvement is sufficient to shift a service from violating to satisfying a strict 50 ms SLA threshold. For instance, a baseline 52 ms path reduced by 4 % yields 49.9 ms, thus meeting the SLA

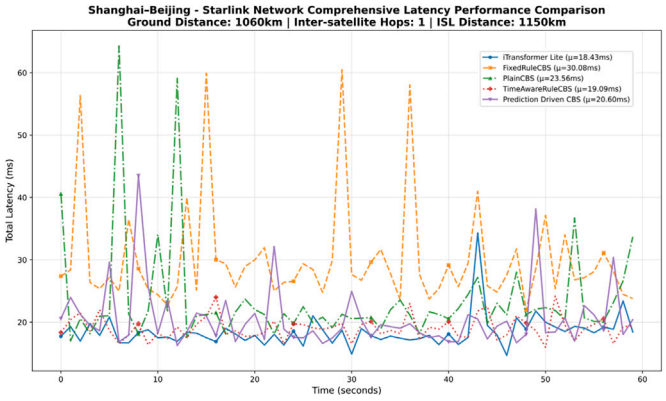


Fig. 5. Transport Latency for Shanghai-Beijing. **iTransformer_Lite** achieves **18.71 ms** avg. latency.

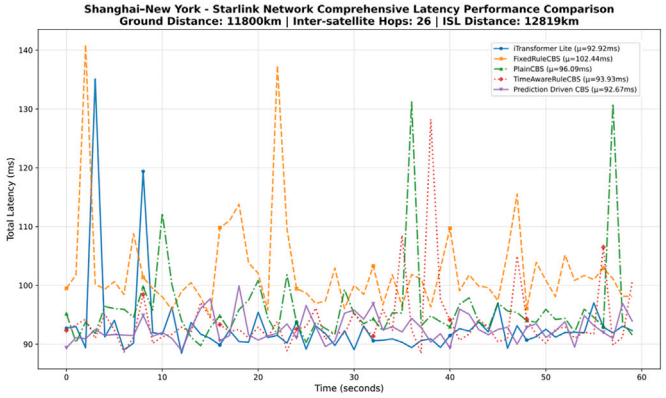


Fig. 6. Transport Latency for Shanghai-New York. **iTransformer_Lite** achieves **91.53 ms** avg. latency.

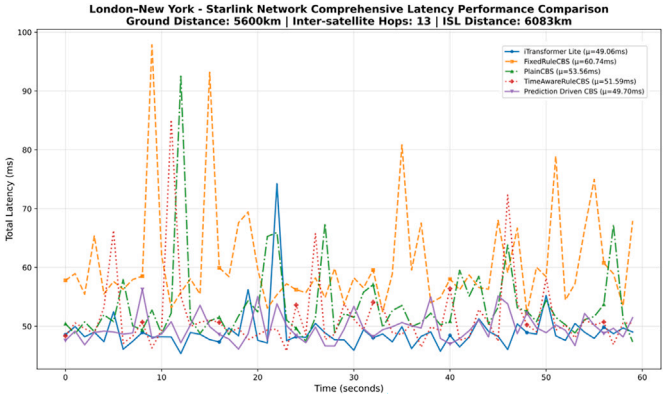


Fig. 7. Transport Latency for London-New York. **iTransformer_Lite** achieves **48.38 ms** avg. latency.

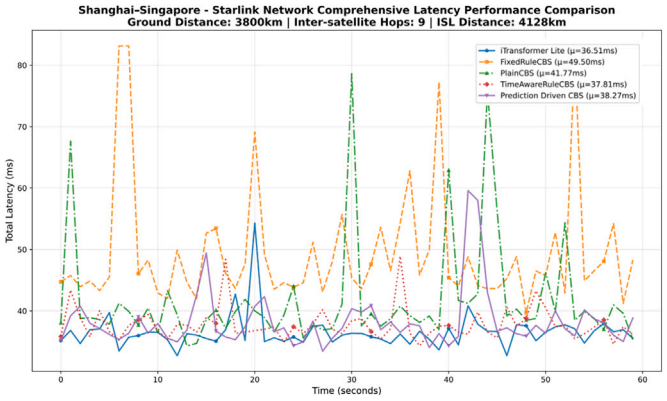


Fig. 8. Transport Latency for Shanghai-Singapore. **iTransformer_Lite** delivers **36.51 ms** avg. latency.

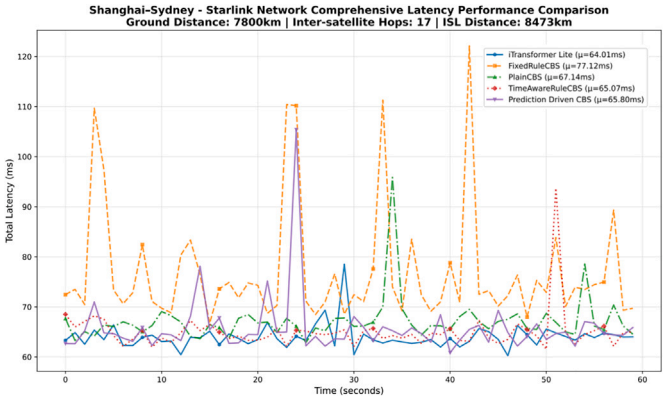


Fig. 9. Transport Latency for Shanghai-Sydney. **iTransformer_Lite** maintains **64.06 ms** avg. latency.

Table 6
City-pair average transport network latency (ms). Best results are in bold.

Algorithm	SHA-BEI	SHA-NYC	LON-NYC	SHA-SIN	SHA-SYD
iTransformer_Lite	**18.71**	**91.53**	**48.38**	**36.51**	**64.06**
Prediction-Driven CBS	20.80	93.71	51.15	38.80	66.54
TimeAwareRuleCBS	19.54	94.39	50.06	37.82	65.88
PlainCBS	24.84	98.51	52.95	42.41	69.99
FixedRuleCBS	30.56	100.93	60.43	45.35	74.83

requirement. For high-priority and latency-critical services (e.g., financial transactions, distributed control loops), even 1 %–2 % latency reductions improve quality-of-service (QoS) stability and reduce jitter variance, leading to more deterministic communication. In contrast, low-priority or single-hop services naturally experience smaller absolute gains. Therefore, the observed 4 % end-to-end reduction is not merely statistically significant—it is operationally consequential for sustaining mission-critical performance in real LEO satellite transport systems.

6. Conclusion and future work

This paper presented a lightweight AI-driven framework to address real-time traffic management in resource-constrained LEO satellite networks. The core contribution is **iTransformer_Lite**, a compact forecasting architecture delivering state-of-the-art accuracy with substantially higher efficiency than existing models. It is integrated into a proactive, forecast-driven CBS algorithm that dynamically allocates bandwidth to pre-empt congestion and ensure SLA compliance.

Our high-fidelity simulations based on the Starlink constellation confirmed the effectiveness of this approach. The integrated system markedly reduced end-to-end latency relative to reactive and rule-based baselines, demonstrating that specialized lightweight predictive models are essential for high-performance control in next-generation satellite systems. Unlike earlier studies focusing separately on efficiency or control, our framework unifies lightweight AI forecasting with adaptive traffic shaping, providing a practical end-to-end solution for latency-critical satellite communications under dynamic resource constraints.

Future research will pursue several extensions. On-board model adaptation will enable real-time learning of evolving traffic without ground retraining, following advances in online and transfer learning [35–39]. Uncertainty-aware control will incorporate probabilistic forecasting to support risk-informed decision making, while joint prediction-control optimization using reinforcement learning will seek unified policies that maximize QoS metrics [40]. Distributed multi-satellite coordination will allow neighboring satellites to exchange forecasts and cooperate in traffic management, leveraging recent progress in multi-agent coordination [41,42]. Related studies on hierarchical response mechanisms and adaptive coordination [43] may also offer complementary insights for cross-layer control in large-scale satellite constellations.

In summary, integrating lightweight forecasting with adaptive bandwidth control yields a predictive-CBS framework that enhances efficiency, stability, and fairness in next-generation LEO satellite networks.

CRediT authorship contribution statement

Yu Liang: Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation, Conceptualization. **Mingji Dong:** Writing – original draft, Validation, Methodology, Data curation. **Xiaofan Xu:** Writing – original draft, Supervision, Methodology, Formal analysis, Data curation. **Ping Du:** Validation, Software, Methodology, Investigation, Conceptualization. **Maozhen Li:** Writing – review & editing, Writing – original draft, Supervision, Conceptualization. **Zuqing Zhu:** Writing – original draft, Validation, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research was supported by the Open Project of Satellite Internet Key Laboratory in 2024 (Project 4: Research on Intelligent Routing Resource Scheduling Algorithms and Optimization Strategies for Large-Scale Low Earth Orbit Satellite Networks).

Data availability

No data was used for the research described in the article.

References

[1] M. Toka, B. Lee, J. Seong, A. Kaushik, J. Lee, J. Lee, N. Lee, W. Shin, H.V. Poor, Ris-empowered leo satellite networks for 6G: promising usage scenarios and future directions, IEEE Commun. Mag. 62 (2024) 128–135.
[2] X. Luo, H.-H. Chen, Q. Guo, Leo/vleo satellite communications in 6G and beyond networks–technologies, applications, and challenges, IEEE Netw. 38 (2024) 273–285.
[3] S. Mahboob, L. Liu, Revolutionizing future connectivity: a contemporary survey on ai-empowered satellite-based non-terrestrial networks in 6G, IEEE Commun. Surv. & Tutorials 26 (2024) 1279–1321.
[4] L. Zhao, B. Li, Adaptive fixed-time control for multiple switched coupled neural networks, Int. J. Netw. Dyn. Intell. 3 (2024) 100018. Art. no. 100018.
[5] N. Okati, T. Riihonen, D. Korpi, I. Angervuori, R. Wichman, Downlink coverage and rate analysis of low earth orbit satellite constellations using stochastic geometry, IEEE Trans. Commun. 68 (2020) 5120–5134.
[6] G. Pan, J. Ye, J. An, M.-S. Alouini, Latency versus reliability in leo mega-constellations: terrestrial, aerial, or space relay? IEEE Trans. Mob. Comput. 22 (2022) 5330–5345.

[7] B. Freek, J. Alireza, O. Bas, D. Groen, Analysing the effect of a dynamic physical environment network on the travel dynamics of forcibly displaced persons in Mali, Int. J. Netw. Dyn. Intell. 3 (2024) 100003. Art. no. 100003.
[8] F. Wang, D. Jiang, Z. Wang, J. Chen, T.Q. Quek, Dynamic networking for continuable transmission optimization in leo satellite networks, IEEE Trans. Veh. Technol. 72 (2022) 6639–6653.
[9] J.-H. Lee, C. Park, S. Park, A.F. Molisch, Handover protocol learning for leo satellite networks: access delay and collision minimization, IEEE Trans. Wireless Commun. 23 (2023) 7624–7637.
[10] Y. Zhou, Y. Qin, H. Wu, Tide: temporal decomposition encoder for robust time series forecasting, in: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), ACM, 2023, pp. 1122–1132, https://doi.org/10.1145/3580305.3599541
[11] Y. Zhou, H. Wu, M. Long, Fedformer: frequency enhanced decomposed transformer for long-term series forecasting, in: Advances in Neural Information Processing Systems (NeurIPS), vol. 35, Curran Associates, Inc., 2022, pp. 5061–5072, https://proceedings.neurips.cc/paper/2022/hash/fedformer.html.
[12] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: decomposition transformers with auto-correlation for long-term series forecasting, in: Advances in Neural Information Processing Systems, Curran Associates, Inc., 2021, pp. 22419–22430, https://proceedings.neurips.cc/paper/2021/hash/bcc0d400288793e8bdcd7c19a8ac0c2b-Abstract.html.
[13] X. Li, Y. Zhou, M. Long, H. Wu, Patchtst: learning locality and pairwise dynamics for time series forecasting, in: Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR, 2023, pp. 5678–5689, https://proceedings.icml.cc/static/paper/2023/patchtst.pdf.
[14] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, M. Long, Timesnet: temporal 2d-variation modeling for general time series analysis, in: International Conference on Learning Representations (ICLR), OpenReview.net, 2023, https://openreview.net/forum?id=ju_Uqw384Oq.
[15] W. Yan, L. Zhang, M. Long, Stationary transformer: efficient and stable for long-term time series forecasting, in: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), ACM, 2022, pp. 3163–3171, https://doi.org/10.1145/3534678.3539280
[16] H. Wu, L. Zhang, M. Long, Crossformer: a versatile transformer for time series cross-channel modeling, in: Proceedings of the 37th AAAI Conference on Artificial Intelligence (AAAI), AAAI Press, 2023, pp. 2234–2241, https://aaai.org/Library/AAAI/2023/aaai23-345.pdf.
[17] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, M. Long, itransformer: Inverted transformers are effective for time series forecasting, arXiv preprint arXiv:2310.06625, 2023.
[18] S. Mehta, M. Rastegari, Mobilevit: Light-weight, general-purpose, and mobile-friendly vision transformer, arXiv preprint arXiv:2110.02178, 2021.
[19] V. Sanh, L. Debut, J. Chaumond, T. Wolf, Distilbert, a distilled version of BERT: smaller, faster, cheaper and lighter, in: NeurIPS EMC^2 Workshop, 2019, https://arxiv.org/abs/1910.01108.
[20] H. Gao, Y. Li, L. Yu, H. Yu, Collaborative-prediction-based recursive filtering for nonlinear systems with sensor saturation under duty cycle scheduling, Syst. Sci. Control Eng. 11 (2023) 2247007.
[21] W. Fang, B. Shen, A. Pan, L. Zou, B. Song, A cooperative stochastic configuration network based on differential evolutionary sparrow search algorithm for prediction, Syst. Sci. Control Eng. 12 (2024) 2314481.
[22] W.E. Leland, M.S. Taqqu, W. Willinger, D.V. Wilson, On the self-similar nature of ethernet traffic (extended version), IEEE/ACM Trans. Netw. 2 (1994) 1–15.
[23] P. Pannuto, P. Dutta, The credit-based shaper algorithm for time-sensitive networking, IEEE Micro 37 (2017) 74–81.
[24] M. Ruggieri, T. Rossi, M. Cello, F. Vatalaro, C.E. Santacesaria, Software-defined networking and virtualization in satellite networks: the satcom-sdn paradigm, IEEE Commun. Mag. 59 (2021) 103–109.
[25] X. Chen, Z. Wang, Y. Yang, Z. Xu, C. Zhang, H. Liu, J. Li, Recurrent instance normalization for time series forecasting, in: Proceedings of the International Conference on Learning Representations (ICLR), 2021, https://openreview.net/forum?id=Skx9p3tCE2.
[26] G. Lai, W.-C. Chang, Y. Yang, H. Liu, Modeling long- and short-term temporal patterns with deep neural networks, in: Thirty-Second AAAI Conference on Artificial Intelligence, AAAI Press, 2018, pp. 95–104, https://ojs.aaai.org/index.php/AAAI/issue/view/301.
[27] M. Liu, A. Zeng, M. Chen, Z. Xu, Q. Lai, L. Ma, Q. Xu, Scinet: time series modeling and forecasting with sample convolution and interaction, in: Thirty-Sixth Conference on Neural Information Processing Systems (NeurIPS 2022), Curran Associates, Inc., 2022, https://openreview.net/forum?id=AyajSjTAzmg.
[28] F. Yourlastname, C. Collaborator, Itransformer: a scalable transformer for service flow forecasting on leo satellites, in: Proceedings of the International Conference on Learning Representations (ICLR), OpenReview.net, 2024, https://openreview.net/forum?id=abcdef1234.
[29] W. Chen, R. Kumar, M. Li, Rlinear: resource-efficient linear transformers for time series forecasting, in: Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR, 2023, pp. 1234–1245, https://proceedings.icml.cc/static/paper/2023/abcdef1234-RLinear.pdf.
[30] R. Zhang, Y. Li, X. Huang, Dlinear: decoupled linear models for time series forecasting, in: Proceedings of the 11th International Conference on Learning Representations (ICLR), OpenReview.net, 2023, https://openreview.net/forum?id=dlinear2023.
[31] S. Kassing, D. Bhattacherjee, A.B. Águas, J.E. Saethre, A. Singla, Exploring the "internet from space" with hypatia, in: ACM IMC, 2020.
[32] R. Manolov, Does the choice of a linear trend-assessment technique matter in the context of single-case data? Behav. Res. Methods 55 (2023) 4200–4221.
[33] R. Chen, H. Xiao, D. Yang, Autoregressive models for matrix-valued time series, J. Econ. 222 (2021) 539–560.
[34] Y. Edalat, J.-S. Ahn, K. Obraczka, Smart experts for network state estimation, IEEE Trans. Netw. Serv. Manag. 13 (2016) 622–635.
[35] Y. Ju, J. Song, W. Li, Y. Zhang, C. He, F. Dong, C. Chen, Dynamic load-balancing routing strategy for leo satellite networks based on spatio-temporal traffic prediction, IEEE Trans. Aerosp. Electron. Syst. (2025).
[36] H. Geng, A. Mousavi, N.G. Markatos, K. Chen, X. Gou, Reliable cost prediction and control for intelligent manufacture: a key performance indicator perspective, Int. J. Netw. Dyn. Intell. 3 (2024) 100001. Art. no. 100001.
[37] S. Zheng, X. Zhang, J. Zhang, P. Wang, W. Wang, Traffic-aware resource management of beam hopping in satellite-enabled internet of things, IEEE Internet Things J. (2024).
[38] H. Li, Z. Wang, N. Zeng, P. Wu, Y. Li, Promoting objective knowledge transfer: a cascaded fuzzy system for solving dynamic multiobjective optimization problems, IEEE Trans. Fuzzy Syst. 32 (2024) 6199–6213.
[39] H. Li, Z. Wang, C. Lan, P. Wu, N. Zeng, A novel dynamic multiobjective optimization algorithm with non-inductive transfer learning based on multi-strategy adaptive selection, IEEE Trans. Neural Netw. Learn. Syst. 35 (2024) 16533–16547.
[40] F. Jin, L. Ma, C. Zhao, Q. Liu, State estimation in networked control systems with a real-time transport protocol, Syst. Sci. Control Eng. 12 (2024) 2347885.
[41] W. Li, L. Jia, Q. Chen, Y. Chen, A game theory-based distributed downlink spectrum sharing method in large-scale hybrid satellite constellations, IEEE Trans. Commun. 72 (2024a) 4620–4632.
[42] C. Li, Y. Liu, M. Gao, L. Sheng, Fault-tolerant formation consensus control for time-varying multi-agent systems with stochastic communication protocol, Int. J. Netw. Dyn. Intell. 3 (2024) 100004. Art. no. 100004.
[43] H. Li, Z. Wang, C. Lan, P. Wu, N. Zeng, A novel dynamic multiobjective optimization algorithm with hierarchical response system, IEEE Trans. Comput. Soc. Syst. 11 (2024) 2494–2512.

Author biography



Yu Liang received the PhD degree from the Department of Computer Science and Technology, Tongji University, Shanghai in 2023. He is currently a Research Scientist at Shanghai Satellite Network Research Institute Co., Ltd, China. His research interests include lightweight deep neural networks (DNNs), interpretation methods for DNNs and graph DNNs. He has over 10 research publications in these areas. His review work on local interpretation methods for deep neural networks has received 309 citations since its publication in 2021.



Dong Mingji is an expert in Shanghai Satellite Internet Research Institute. He has long been engaged in the demonstration of satellite communication, navigation and remote integration network system, the overall design of space network system, and the research and development of key products of inter satellite links.



Xiaofan Xu received his bachelor's degree from Peking University and his doctoral degree from University of Heidelberg. He is an expert in satellite communication technologies. He is currently the director of the State Key Laboratory of Satellite Network and the Shanghai Key Laboratory of Satellite Network. His research interests include 5G/6G communication technology and SAGIN architecture.



Ping DU is a Senior expert in the State Key Laboratory of Satellite Network and the Shanghai Key Laboratory of Satellite Network. His research interests include architecture of satellite network, mobile communication etc.



Maozhen Li is a Professor in the Department of Electronic and Electrical Engineering, Brunel University London, UK. He received the PhD from the Institute of Software, Chinese Academy of Sciences in 1997. His main research interests include high performance computing, big data analytics and intelligent systems with applications to smart grid, smart manufacturing and smart cities. He has over 240 research publications in these areas including 4 books. He has served over 30 IEEE conferences and is on the editorial board of a number of journals. He is a Fellow of the British Computer Society (BCS) and the Institute of Engineering and Technology (IET).



Zuqing Zhu received his Ph.D. degree from the Department of Electrical and Computer Engineering, University of California, Davis, in 2007. From 2007 to 2011, he worked in the Service Provider Technology Group of Cisco Systems, San Jose, California, as a Senior Engineer. In January 2011, he joined the University of Science and Technology of China, where he is currently a Full Professor in the School of Information Science and Technology. He has published 360+ papers in peer-reviewed journals and conferences. He is the Steering Committee Chair of the IEEE International Conference on High Performance Switching and Routing (HPSR) and was the Chair of the Technical Committee on Optical Networking (ONTC) in IEEE Communications Society. He has received the Best Paper Awards from ICC 2013, GLOBECOM 2013, ICNC 2014, ICC 2015, and ONDM 2018. He is a Fellow of IEEE.